

Cómo crear un problema para DOMjudge

Guía paso a paso, con un ejemplo completo de principio a fin

Paolo Boris Luna Luque

IEEE Computer Society - Rama Estudiantil

Julio 2026

EPIS — Universidad Nacional del Altiplano (UNAP)

Este documento explica, con un ejemplo real de principio a fin, cómo preparar un problema de programación competitiva — enunciado, solución de referencia, casos de prueba, empaquetado y subida — para usarlo en un concurso organizado con DOMjudge.

Índice

1. Introducción a DOMjudge y Entornos de Competencia	2
2. ¿Qué es un “problema” en DOMjudge?	2
3. Anatomía de la carpeta de un problema	3
4. Ejemplo completo: “Contar números pares”	4
4.1. Paso 1 — Redactar el enunciado	4
4.2. Paso 2 — Crear la estructura de carpetas	4
4.3. Paso 3 — Escribir <code>problem.yaml</code>	4
4.4. Paso 4 — Escribir <code>domjudge-problem.ini</code>	5
4.5. Paso 5 — Escribir la solución de referencia	5
4.6. Paso 6 — Generar los casos de prueba	5
4.7. Paso 7 — Comprimir el problema	8
4.8. Paso 8 — Importar en DOMjudge si tienes acceso	8
4.9. Paso 9 — Probar el problema	9
5. Plantilla en blanco para tu propio problema	10
6. Errores comunes al crear problemas	11
7. Conclusión	11

1 Introducción a DOMjudge y Entornos de Competencia

En el ámbito de la programación competitiva de alto nivel —marcado por competencias internacionales emblemáticas como el **ICPC** (International Collegiate Programming Contest) y el **IEEEExtreme**—, la evaluación de las soluciones entregadas por los participantes debe ser automática, determinista, transparente y en tiempo real.

Para lograr este estándar de precisión, la plataforma preferida por la comunidad científica y las universidades de todo el mundo es **DOMjudge**. Se trata de un sistema de juicio en línea (*Automated Judge System*) altamente escalable, diseñado específicamente para gestionar concursos de programación bajo las reglas de la ACM-ICPC.

En este entorno, un concurso no es solo un conjunto de algoritmos ejecutándose en un servidor, sino una infraestructura rigurosa donde cada milisegundo de ejecución y cada byte de memoria cuentan. Por ello, el concepto de “problema” no se limita al documento PDF que describe una historia o un reto matemático; desde la perspectiva de la plataforma, un problema es un paquete de datos estandarizado que contiene las especificaciones técnicas necesarias para validar la corrección de un algoritmo.

2 ¿Qué es un “problema” en DOMjudge?

Un problema, desde el punto de vista del sistema (no del enunciado que lee el participante), es una carpeta con una estructura fija que contiene:

- Un archivo de configuración con el nombre del problema y el límite de tiempo.
- Un conjunto de **casos de prueba**: pares de archivos entrada/salida esperada.
- Opcionalmente, el enunciado en sí (aunque, como se explica más adelante, es más confiable pegarlo directamente en el panel web).

Todo esto se comprime en un archivo `.zip` y se sube al sistema desde **Import** / **export** → **Problems** → **Import archive**.

Nota

No hace falta escribir el juez (*judge*) que compara las respuestas — DOMjudge ya lo trae incluido. Por defecto compara la salida del programa del participante **exactamente** contra el archivo de salida esperada (ignorando espacios en blanco al final de línea y saltos de línea finales).

3 Anatomía de la carpeta de un problema

Listing 1: Estructura de carpetas (antes de comprimir)

```

1 mi_problema/
2 |-- problem.yaml
3 |-- domjudge-problem.ini
4 |-- data/
5     |-- sample/
6         |-- 1.in
7         |-- 1.ans
8     |-- secret/
9         |-- 1.in
10        |-- 1.ans
11        |-- 2.in
12        |-- 2.ans
13        |-- 3.in
14        |-- 3.ans

```

Elemento	Descripción
problem.yaml	Nombre del problema y límite de tiempo, en formato YAML (formato moderno, el que usa DOMjudge por defecto).
domjudge-problem.ini	El mismo dato, en formato antiguo tipo “ini”. Se recomienda incluir ambos por compatibilidad entre versiones.
data/sample/	Casos de prueba visibles para el participante — son los que se muestran en el enunciado como ejemplo.
data/secret/	Casos de prueba ocultos , usados para calificar de verdad. El participante nunca los ve.
N.in / N.ans	Cada caso de prueba es un par de archivos con el mismo número: N.in es la entrada, N.ans es la salida esperada exacta para esa entrada.

Atención

Al comprimir, los archivos `problem.yaml`, `domjudge-problem.ini` y la carpeta `data/` deben quedar en la **raíz** del `.zip` — no dentro de una carpeta contenedora adicional (por ejemplo, `mi_problema/problem.yaml` dentro del zip está **mal**; debe ser `problem.yaml` directo en la raíz). Este es el error más común al importar un problema.

4 Ejemplo completo: “Contar números pares”

Vamos a crear, de principio a fin, un problema real llamado “**Contar números pares**”, de dificultad fácil, siguiendo cada paso.

4.1 Paso 1 — Redactar el enunciado

Paso 1: Enunciado

Se escribe primero en un documento aparte (o directamente en el campo de texto del panel de DOMjudge más adelante). Debe incluir: descripción del problema, formato exacto de entrada, formato exacto de salida, restricciones (límites de los valores), y al menos un ejemplo.

Enunciado: Contar números pares

Contar números pares (Fácil)

Dada una lista de n números enteros, cuenta cuántos de ellos son pares.

Entrada: la primera línea contiene un entero n ($1 \leq n \leq 1000$). La segunda línea contiene n enteros separados por espacio ($-10^9 \leq \text{cada valor} \leq 10^9$).

Salida: un entero, la cantidad de números pares en la lista.

Ejemplo

Entrada:

```
6
4 7 2 9 10 3
```

Salida:

```
3
```

4.2 Paso 2 — Crear la estructura de carpetas

Desde una terminal (Linux/Mac) o PowerShell (Windows):

Listing 2: Linux / Mac / Git Bash

```
1 mkdir -p contarpares/data/sample
2 mkdir -p contarpares/data/secret
3 cd contarpares
```

Listing 3: Windows PowerShell (equivalente)

```
1 New-Item -ItemType Directory -Force contarpares\data\sample
2 New-Item -ItemType Directory -Force contarpares\data\secret
3 Set-Location contarpares
```

4.3 Paso 3 — Escribir problem.yaml

Crear el archivo `problem.yaml` en la raíz de `contarpares/`:

Listing 4: problem.yaml

```
1 name: 'Contar numeros pares'
2 timelimit: 1
```

4.4 Paso 4 — Escribir domjudge-problem.ini

Listing 5: domjudge-problem.ini

```
1 probid = contarparees
2 name = Contar numeros pares
3 timelimit = 1
```

Consejo

El valor de `probid` debe ser corto, sin espacios ni acentos (solo letras minúsculas y números) — se usa internamente como identificador único del problema dentro del concurso.

4.5 Paso 5 — Escribir la solución de referencia

Esta solución **no** se sube al sistema ni la ven los participantes — se usa únicamente para generar las respuestas correctas de los casos de prueba, de forma automática y sin errores humanos de cálculo.

Listing 6: solucion.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     int n;
6     cin >> n;
7     int pares = 0;
8     for(int i = 0; i < n; i++){
9         long long x;
10        cin >> x;
11        if (x % 2 == 0) pares++;
12    }
13    cout << pares << "\n";
14    return 0;
15 }
```

Se compila una sola vez:

```
1 g++ -O2 -o solucion solucion.cpp
```

4.6 Paso 6 — Generar los casos de prueba

Nota

Antes de seguir, lo más importante de esta sección: **cada caso de prueba son dos archivos de texto plano separados** — por ejemplo `1.in` y `1.ans` — que solo contienen números o texto simple, **nunca código**. No es un solo archivo, no es un programa, y no es Python: van a ser 12 archivos de texto individuales en total para este ejemplo (6 archivos `.in` + 6 archivos `.ans`). El único lugar donde sí se usa un poco de Python es al final, para un único caso (el de 1000 números), y es completamente opcional — se explica más abajo por qué.

6.1 —Cuál es el contenido exacto de cada archivo

La siguiente tabla muestra, archivo por archivo, exactamente qué texto debe quedar guardado adentro. Esto es el resultado final que se debe tener antes de comprimir — todavía no importa cómo se creó cada archivo, eso se explica después.

Archivo	Contenido (texto plano)	Qué representa
data/sample/1.in	6 4 7 2 9 10 3	Caso de ejemplo (el mismo del enunciado)
data/sample/1.ans	3	Respuesta correcta de ese caso
data/secret/1.in	4 1 3 5 7	Caso oculto: todos impares
data/secret/1.ans	0	Respuesta correcta
data/secret/2.in	4 2 4 6 8	Caso oculto: todos pares
data/secret/2.ans	4	Respuesta correcta
data/secret/3.in	1 0	Caso oculto: un solo elemento (el 0 cuenta como par)
data/secret/3.ans	1	Respuesta correcta
data/secret/4.in	5 -2 -3 -4 -5 -6	Caso oculto: números negativos
data/secret/4.ans	3	Respuesta correcta
data/secret/5.in	1000 (1000 números separados por espacio, generados al azar)	Caso oculto: tamaño grande, al límite permitido
data/secret/5.ans	(la cantidad de pares entre esos 1000 números)	Respuesta correcta

Nunca se escribe el .ans a mano (ni siquiera para los casos pequeños) — siempre se obtiene ejecutando la solución de referencia (`solucion.cpp` ya compilado, del Paso 5) con ese `.in` como entrada, y guardando lo que imprime como el `.ans`. Esto evita errores de cálculo humano.

6.2 — Opción A: crearlos por terminal (rápido)

Si se siente cómodo usando la terminal, cada par de archivos se crea con dos líneas: una que escribe el `.in`, y otra que ejecuta la solución leyendo ese `.in` y guardando su salida como el `.ans` correspondiente.

Listing 7: Caso de ejemplo (visible) — crea 2 archivos: 1.in y 1.ans

```
1 printf "6\n4 7 2 9 10 3\n" > data/sample/1.in
2 ./solucion < data/sample/1.in > data/sample/1.ans
```

Listing 8: Casos ocultos — cada bloque de 2 líneas crea un par `.in/.ans` distinto

```
1 # Caso oculto 1: todos impares -> crea data/secret/1.in y 1.ans
2 printf "4\n1 3 5 7\n" > data/secret/1.in
3 ./solucion < data/secret/1.in > data/secret/1.ans
4
5 # Caso oculto 2: todos pares -> crea data/secret/2.in y 2.ans
6 printf "4\n2 4 6 8\n" > data/secret/2.in
7 ./solucion < data/secret/2.in > data/secret/2.ans
8
9 # Caso oculto 3: un solo elemento -> crea data/secret/3.in y 3.ans
10 printf "1\n0\n" > data/secret/3.in
11 ./solucion < data/secret/3.in > data/secret/3.ans
12
13 # Caso oculto 4: numeros negativos -> crea data/secret/4.in y 4.ans
```

```
14 printf "5\n-2 -3 -4 -5 -6\n" > data/secret/4.in
15 ./solucion < data/secret/4.in > data/secret/4.ans
```

Nota

`printf "6\n4 7 2 9 10 3\n"` no es código ni una fórmula — es simplemente la forma en que la terminal escribe texto en un archivo. `\n` significa “salto de línea”. Ese comando produce exactamente el mismo archivo `1.in` de dos líneas (6 y luego 4 7 2 9 10 3) que se muestra en la tabla de arriba — es solo una forma más rápida de escribirlo que abrir un editor de texto.

6.3 — Opción B: crearlos sin terminal, solo con pantalla (Bloc de notas)

Si se prefiere no usar la terminal, se puede hacer exactamente lo mismo manualmente, archivo por archivo, con cualquier editor de texto simple (por ejemplo, el Bloc de notas de Windows):

1. Abrir el Bloc de notas (Notepad).
2. Escribir el contenido exacto del archivo `.in` según la tabla de la sección 6.1 (por ejemplo, para `data/sample/1.in`: escribir 6 en la primera línea, y 4 7 2 9 10 3 en la segunda línea).
3. Ir a **Archivo** → **Guardar como**.
4. Navegar hasta la carpeta `contarpares/data/sample/` (creada en el Paso 2).
5. En “Nombre de archivo” escribir exactamente `1.in` (con las comillas si el cuadro lo pide, para que no agregue `.txt` al final: `"1.in"`). En “Tipo” elegir **Todos los archivos** en vez de “Documentos de texto”.
6. Guardar.
7. Para generar el `.ans` correspondiente: abrir una terminal (aunque sea solo para este paso) y ejecutar el programa `solucion` con ese archivo como entrada, por ejemplo: `solucion.exe <data\sample\1.in` — el número que aparece en pantalla es la respuesta correcta. Copiar ese número, abrir un nuevo Bloc de notas, pegarlo, y guardarlo como `1.ans` en la misma carpeta (mismo proceso de “Guardar como”).
8. Repetir estos mismos pasos para cada uno de los 5 casos ocultos restantes, usando el contenido de la tabla de la sección 6.1 y guardando cada uno en `data/secret/` con su número correspondiente (`1.in/1.ans`, `2.in/2.ans`, etc.).

Atención

El paso de generar el `.ans` **siempre** requiere ejecutar el programa `solucion` una vez por cada `.in` — no hay forma de evitar usar la terminal para ese paso puntual, aunque todo lo demás (escribir los `.in`, guardar los `.ans`) se haga solo con el Bloc de notas.

6.4 — Sobre el caso grande (1000 números) y el script de Python

El último caso oculto (`data/secret/5.in`) necesita 1000 números dentro de una sola línea, para probar que la solución de los participantes también funciona rápido con una entrada grande (no solo con ejemplos pequeños). Escribir 1000 números a mano no es práctico, así que — solo para **este** caso — se usa un pequeño script en un lenguaje puede ser de Python u otro que los genera al azar automáticamente:

Listing 9: Solo para el caso 5: genera 1000 numeros al azar ->data/secret/5.in

```

1 python3 -c "
2 import random
3 n = 1000
4 print(n)
5 print(' '.join(str(random.randint(-10**9, 10**9)) for _ in range(n)))
6 " > data/secret/5.in
7 ./solucion < data/secret/5.in > data/secret/5.ans

```

Consejo

Este script es **opcional** y solo se usa para este caso puntual — el resto del problema no necesita Python en absoluto. Si no se quiere instalar Python, es perfectamente válido reemplazar este caso por una lista más corta escrita a mano (por ejemplo, 20 números en vez de 1000); simplemente ese caso no probará el límite máximo de tamaño con la misma exigencia.

Consejo

Buenos casos de prueba cubren: el caso típico del enunciado, valores extremos (mínimo y máximo permitido), tamaños límite ($n = 1$ y n máximo), y casos que tienden a romper soluciones ingenuas (todos iguales, todos en un extremo, negativos si están permitidos, etc.).

4.7 Paso 7 — Comprimir el problema

Listing 10: Linux / Mac / Git Bash — IMPORTANTE: pararse dentro de la carpeta

```

1 cd contarpares
2 zip -r ../contarpares.zip problem.yaml domjudge-problem.ini data

```

Listing 11: Windows PowerShell

```

1 Set-Location contarpares
2 Compress-Archive -Path problem.yaml,domjudge-problem.ini,data '
3 -DestinationPath ../contarpares.zip -Force

```

Verificar el contenido antes de subirlo (los archivos deben verse en la raíz, sin ninguna carpeta `contarpares/` adicional dentro del zip):

```

1 unzip -l ../contarpares.zip

```

Atención

Si al hacer clic derecho en Windows y usar “Enviar a → Carpeta comprimida” sobre la *carpeta* `contarpares` completa, el zip resultante tendrá una carpeta `contarpares/` por dentro — eso **rompe la importación**. Hay que seleccionar los *archivos y carpetas de adentro* (`problem.yaml`, `domjudge-problem.ini`, `data`) y comprimir esos directamente, no la carpeta que los contiene.

4.8 Paso 8 — Importar en DOMjudge si tienes acceso

1. Entrar al panel de administración con la cuenta `admin`.
2. Ir a **Import** / **export**.

3. En la sección **Problems**, elegir el **Contest** correcto (el concurso al que pertenece este problema).
4. En **Import archive**, seleccionar el archivo `contar pares.zip`.
5. Hacer clic en **Import**.
6. Ir a **Problems** en el menú, buscar “Contar numeros pares” en la lista, y entrar a editarlo.
7. Pegar el enunciado (el texto del Paso 1) en el campo de texto del problema, guardar.

4.9 Paso 9 — Probar el problema

Paso 9: Verificación

Nunca dar un problema por terminado sin probarlo con al menos dos envíos: uno que debería ser **Accepted** (con la misma solución de referencia u otra correcta) y uno que debería fallar a propósito (por ejemplo, un código con un error deliberado), para confirmar que el juez realmente distingue una respuesta correcta de una incorrecta.

1. Iniciar sesión con una cuenta de equipo de prueba (por ejemplo, `team01`).
2. Ir a **Submit**, elegir el problema “Contar numeros pares” y el lenguaje correspondiente.
3. Subir el código `solucion.cpp` (la misma solución de referencia) y enviar.
4. Confirmar que el veredicto es **Correct/Accepted**.
5. Enviar una segunda solución con un error a propósito (por ejemplo, contar los impares en vez de los pares) y confirmar que el veredicto es **Wrong Answer**.

5 Plantilla en blanco para tu propio problema

Copia esta plantilla y llénala para crear un problema nuevo, reemplazando cada parte marcada con `<...>`.

Listing 12: Plantilla de enunciado

```

1 <Nombre del problema> (<Facil / Facil-Intermedio / Intermedio / Dificil>)
2
3 <Descripcion del problema en 2-4 lineas>
4
5 Entrada: <formato exacto de la entrada, con limites de cada valor>
6 Salida: <formato exacto de la salida>
7
8 Ejemplo
9 Entrada:
10 <entrada de ejemplo>
11 Salida:
12 <salida de ejemplo>

```

Listing 13: Plantilla problem.yaml

```

1 name: '<Nombre del problema>'
2 timelimit: <segundos, normalmente 1 o 2>

```

Listing 14: Plantilla domjudge-problem.ini

```

1 probid = <id_corto_sin_espacios>
2 name = <Nombre del problema>
3 timelimit = <segundos>

```

Checklist antes de importar	Listo
¿El enunciado especifica los límites exactos de cada valor de entrada?	☐
¿Hay al menos un caso de ejemplo en <code>data/sample/</code> que coincide con el enunciado?	☐
¿Hay casos ocultos en <code>data/secret/</code> cubriendo casos límite (mínimo, máximo, negativos, empates, etc.)?	☐
¿Cada <code>.ans</code> fue generado ejecutando la solución de referencia (no calculado a mano)?	☐
¿ <code>problem.yaml</code> y <code>domjudge-problem.ini</code> están en la raíz del zip, sin carpeta contenedora?	☐
¿Se importó y se probó con al menos un envío correcto y uno incorrecto?	☐

6 Errores comunes al crear problemas

Síntoma	Causa probable	Solución
Al importar, DOMjudge no reconoce el problema o falla silenciosamente	<code>problem.yaml</code> quedó dentro de una carpeta adicional en el zip	Volver a comprimir parado <i>dentro</i> de la carpeta del problema (Paso 7)
Todas las soluciones correctas dan Wrong Answer	El archivo <code>.ans</code> no coincide exactamente con la salida (por ejemplo, espacio extra, formato de número distinto)	Regenerar el <code>.ans</code> ejecutando la solución de referencia, nunca escribirlo a mano
Una solución incorrecta da Accepted	Faltan casos de prueba que cubran el error específico de esa solución	Agregar un caso oculto diseñado específicamente para detectar ese error
El problema tarda demasiado en juzgar o da Time Limit Exceeded en soluciones correctas	El límite de tiempo (<code>timelimit</code>) es muy ajustado para la complejidad esperada	Aumentar el <code>timelimit</code> en <code>problem.yaml</code> y <code>domjudge-problem.ini</code> , o revisar la solución de referencia
El enunciado no aparece o aparece vacío tras importar	El zip no incluía un enunciado reconocible, o el formato no se procesó	Pegar el enunciado manualmente en el campo de texto del problema, desde Problems en el panel (Paso 8, punto 7)

7 Conclusión

Crear un problema de programación competitiva siempre sigue el mismo patrón: enunciado claro con límites exactos, una solución de referencia confiable, un conjunto de casos de prueba (ejemplo + ocultos) generados automáticamente a partir de esa solución, y una verificación final con al menos un envío correcto y uno incorrecto antes de darlo por terminado. Siguiendo el ejemplo de “Contar números pares” de este documento paso a paso, cualquier persona del equipo organizador puede preparar un problema nuevo de forma consistente.